

```
In [1]: import torch
import torchvision
from torch import nn
from torch.utils import data
from torchvision import transforms
import matplotlib.pyplot as plt
```

```
In [2]: def get_dataloader_workers():
        """使用 4 个子进程并行读取数据。"""
        return 4
```

```
In [3]: def load_data_fashion_mnist(batch_size, resize=None): # resize=None 表示是否调整图片大小
        """下载Fashion-MNIST数据集，然后将其加载到内存中"""
        trans = [transforms.ToTensor()]
        if resize:
            trans.insert(0, transforms.Resize(resize)) # 在转换为 tensor 前先调整大小
        trans = transforms.Compose(trans) # 组合所有的操作
        mnist_train = torchvision.datasets.FashionMNIST(
            root="./data", train=True, transform=trans, download=True)
        mnist_test = torchvision.datasets.FashionMNIST(
            root="./data", train=False, transform=trans, download=True)
        return (data.DataLoader(mnist_train, batch_size, shuffle=True,
                                num_workers=get_dataloader_workers()),
                data.DataLoader(mnist_test, batch_size, shuffle=False,
                                num_workers=get_dataloader_workers()))
```

```
In [4]: batch_size = 256
train_iter, test_iter = load_data_fashion_mnist(batch_size)
```

4.2.1. 初始化模型参数

```
In [5]: num_inputs, num_outputs, num_hiddens = 784, 10, 256

W1 = nn.Parameter(torch.randn(
    num_inputs, num_hiddens, requires_grad=True) * 0.01)
# 创建一个形状为 (num_inputs, num_hiddens) 的张量
# * 0.01 将初始随机值缩小到较小范围
# nn.Parameter(...) 将普通张量包装为模型的可学习参数
b1 = nn.Parameter(torch.zeros(num_hiddens, requires_grad=True))
W2 = nn.Parameter(torch.randn(
    num_hiddens, num_outputs, requires_grad=True) * 0.01)
b2 = nn.Parameter(torch.zeros(num_outputs, requires_grad=True))

params = [W1, b1, W2, b2]
```

4.2.2. 激活函数

```
In [6]: def relu(X):
        a = torch.zeros_like(X)
        return torch.max(X, a)
```

4.2.3. 模型

```
In [7]: def net(X):
        X = X.reshape((-1, num_inputs))
        # num_inputs = 784 = 1 × 28 × 28
        # -1 表示自动计算这一维；展平为 784 个特征
        H = relu(X@W1 + b1) # 这里“@”代表矩阵乘法
        return (H@W2 + b2)
```

4.2.4. 损失函数

```
In [8]: loss = nn.CrossEntropyLoss(reduction='none')
# reduction='none': 不把一个批次中所有样本的损失汇总为一个数，而是保留每个样本各自的损失。
```

4.2.5. 训练

```
In [9]: def accuracy(y_hat, y):
        """计算预测正确的样本数。"""
        # .item(): 把“只包含一个值的 PyTorch 张量”转换为普通 Python 数值。
        return (y_hat.argmax(dim=1) == y).sum().item()
```

```
def evaluate_accuracy(net, data_iter):
    """计算模型在数据集上的准确率。"""
    correct, total = 0, 0
    with torch.no_grad():
        for X, y in data_iter:
            correct += accuracy(net(X), y)
            total += y.numel()
    return correct / total

def train_epoch(net, train_iter, loss, updater):
    """训练模型一个迭代周期，返回平均损失和准确率。"""
    total_loss, correct, total = 0.0, 0, 0 # 损失之和、正确预测数、样本数
    for X, y in train_iter:
        y_hat = net(X)
        l = loss(y_hat, y).mean() # 一个 batch 的平均损失

        updater.zero_grad() # 清空旧梯度
        l.backward() # 反向传播
        updater.step() # 更新参数 parameter -= 0.1 * parameter.grad

        batch_size = y.numel() # 计算当前批次的样本数
        total_loss += l.item() * batch_size # l.item() 是当前批次的平均损失，乘以 batch_size 得到总损失
        correct += accuracy(y_hat, y) # 计算当前批次的正确预测数
        total += batch_size
    return total_loss / total, correct / total
```

```
In [10]: num_epochs, lr = 10, 0.1
        updater = torch.optim.SGD(params, lr=lr) # params = [w1, b1, w2, b2]
```

```
In [11]: for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, updater)
        test_acc = evaluate_accuracy(net, test_iter)
        print(f'epoch {epoch + 1}/{num_epochs}: '
              f'loss {train_loss:.4f}, train acc {train_acc:.3f}, '
              f'test acc {test_acc:.3f}')
```

```
epoch 1/10: loss 1.0367, train acc 0.653, test acc 0.761
epoch 2/10: loss 0.6022, train acc 0.789, test acc 0.779
epoch 3/10: loss 0.5182, train acc 0.820, test acc 0.817
epoch 4/10: loss 0.4782, train acc 0.832, test acc 0.824
epoch 5/10: loss 0.4536, train acc 0.841, test acc 0.826
epoch 6/10: loss 0.4337, train acc 0.848, test acc 0.826
epoch 7/10: loss 0.4170, train acc 0.853, test acc 0.840
epoch 8/10: loss 0.4058, train acc 0.857, test acc 0.826
epoch 9/10: loss 0.3942, train acc 0.861, test acc 0.848
epoch 10/10: loss 0.3838, train acc 0.864, test acc 0.850
```

4.2.6. 预测

```
In [12]: def get_fashion_mnist_labels(labels):
        """返回Fashion-MNIST数据集的文本标签"""
        text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',
                       'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
        return [text_labels[int(i)] for i in labels]
```

```
In [13]: def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5): #@save
        """绘制图像列表"""
        figsize = (num_cols * scale, num_rows * scale) # 计算画布的大小
        _, axes = plt.subplots(num_rows, num_cols, figsize=figsize) # 创建多个图片窗口, axes 表示所有子图对象
        axes = axes.flatten() # 将 axes 展平成一维数组
        for i, (ax, img) in enumerate(zip(axes, imgs)): # zip() 实现配对得到 (ax, img); enumerate() 给列表增加索引
            if torch.is_tensor(img):
                # 图片张量
                ax.imshow(img.numpy()) # matplotlib 只认识 numpy, 不认识 tensor
            else:
                # PIL 图片
                ax.imshow(img)
            # 隐藏坐标轴
            ax.axes.get_xaxis().set_visible(False)
            ax.axes.get_yaxis().set_visible(False)
            # 显示标题
            if titles:
                ax.set_title(titles[i])
        return axes
```

```
In [14]: def predict_ch3(net, test_iter, n=6):
        """预测标签"""
        for X, y in test_iter: # 取一个 batch
```

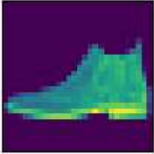
```

break
trues = get_fashion_mnist_labels(y) # 真实标签
preds = get_fashion_mnist_labels(net(X).argmax(axis=1)) # 预测标签
titles = [true + '\n' + pred for true, pred in zip(trues, preds)] # 标题
show_images(X[:n].reshape(n, 28, 28), 1, n, titles=titles[:n]) # 显示图片

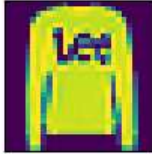
predict_ch3(net, test_iter)

```

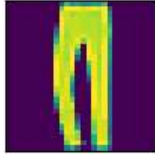
ankle boot
ankle boot



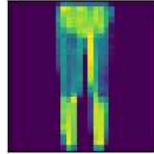
pullover
pullover



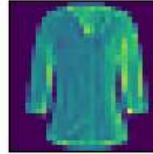
trouser
trouser



trouser
trouser



shirt
shirt



trouser
trouser

